

Software Requirements and Implementation Specification

A precise description of the implemented product, system boundaries, architecture, data model, API, scoring rules, and operational constraints.

Document purpose

This document defines BidSight as it exists in the repository. It is intended for software engineers, reviewers, and technical stakeholders who need to understand the complete system without reading every source file.

Document	Value
Document ID	BS-SPEC-001
Version	1.0
Status	Final
Date	12 August 2026
Prepared by	Badar Butt, AI Engineer

BidSight is a procurement decision-support system. Final purchasing authority remains with the authorised user.

1. Product Definition

BidSight is an AI-assisted vendor quotation evaluation system. It converts text-readable quotation PDFs into structured procurement evidence, places that evidence under human review, evaluates vendors with deterministic Python rules, and uses Microsoft Foundry to write a recommendation grounded in the verified result set.

Problem addressed

Vendor quotations vary in format and terminology. Price, tax, delivery, warranty, payment terms, support, and technical specifications are often distributed across multiple pages. Manual comparison is slow and can make mandatory failures difficult to identify or explain.

System objective

- Reduce repetitive quotation reading and manual data entry.
- Keep extracted facts editable before they become scoring inputs.
- Apply mandatory requirements and score weights consistently.
- Show non-compliant vendors and the reasons they failed.
- Generate an advisory recommendation without delegating the purchasing decision to AI.

Scope boundary

Included	Not included
One procurement evaluation with requirements	Authentication, roles, or approval chains
One to three text-readable PDF quotations	OCR for scanned or image-only documents
Foundry extraction and recommendation	AI-generated compliance or numeric scores
SQLite and local file storage	Production-grade object storage or migrations
Comparison, deletion, CSV export, print report	Purchase order creation or supplier communication

2. End-to-End Workflow

Create evaluation

- > Define mandatory and preferred requirements
- > Upload 1 to 3 vendor PDFs
- > Validate and store each file
- > Extract page text with PyMuPDF
- > Structure quotation data with Microsoft Foundry
- > Review and edit every extracted field
- > Run Python compliance and scoring
- > Compare all vendors
- > Generate a verified-evidence recommendation

Workflow states

State	Meaning
DRAFT	The evaluation has been created.
REQUIREMENTS_READY	Purchasing requirements are available.
QUOTATIONS_UPLOADED	At least one quotation is stored.
REVIEW_REQUIRED	Extraction has completed and requires user confirmation.
READY_FOR_SCORING	All quotations are processed and reviewed.
SCORED	Deterministic comparison results are saved.
RECOMMENDATION_READY	A validated Foundry recommendation is saved.

Result invalidation

Changing requirements, reviewed quotation data, evaluation details, or the vendor set removes saved scores and recommendations. This prevents stale rankings from remaining visible after an input changes.

3. Functional Requirements

ID	Requirement	Implemented behaviour
FR-01	Evaluation management	Create, list, open, update, search, export, and delete evaluations.
FR-02	Requirement management	Capture name, expected value, unit, mandatory or preferred type, and optional operator.
FR-03	Quotation intake	Accept up to three PDFs per evaluation with filename, type, size, and header validation.
FR-04	Document processing	Extract readable page text and submit it to Foundry for structured extraction.
FR-05	Human review	Allow the user to edit extracted commercial and technical fields before scoring.
FR-06	Compliance	Evaluate each requirement as PASS, FAIL, or UNKNOWN.
FR-07	Scoring	Calculate component and overall scores in Python using named weights.
FR-08	Comparison	Show every vendor, including failed and incomplete quotations, with reasons.
FR-09	Recommendation	Generate and validate a structured explanation from verified evidence.
FR-10	Data removal	Delete quotations or complete evaluations and invalidate dependent results.
FR-11	Exports	Download CSV data and print or save the comparison report.
FR-12	Operational feedback	Provide loading, retry, empty, validation, error, and success states.

4. System Architecture

Procurement user

- > Next.js frontend
 - > FastAPI REST API
 - > SQLModel / SQLite
 - > Local PDF storage
 - > PyMuPDF text extraction
 - > Python scoring engine
 - > Microsoft Foundry model deployment

Layer	Technology	Responsibility
Presentation	Next.js, React, TypeScript, Tailwind CSS	Workflow navigation, forms, review, comparison, exports, and user feedback.
API	FastAPI, Pydantic	Request validation, workflow gates, errors, CORS, and response contracts.
Persistence	SQLModel, SQLite	Evaluations, requirements, quotations, results, and saved recommendations.
Documents	PyMuPDF	Safe PDF intake and page-by-page text extraction.
Deterministic logic	Python	Compliance, eligibility, price score, component scores, weighted total, and rank.
Language model	Microsoft Foundry through OpenAI SDK	Structured extraction and final explanation only.

Frontend routes

Route	Purpose
/	Dashboard totals, recent work, and activity.
/evaluations	Evaluation search, filtering, export, open, and deletion.
/evaluations/new	Evaluation and requirement creation.
/evaluations/{id}	Evaluation summary and workflow progress.
/evaluations/{id}/upload	Quotation upload, processing, retry, and removal.
/evaluations/{id}/review	Editable extraction review and confirmation.
/evaluations/{id}/comparison	Scores, compliance, risks, and recommendation.
/vendors	Vendor records derived from quotation data.
/settings	API health, scoring policy, and local settings.

5. Data Model

Entity	Key fields	Purpose
Evaluation	id, title, category, quantity, budget, currency, delivery target, status, recommendation	Defines the purchasing baseline and current workflow state.
Requirement	evaluation_id, name, value, unit, type, operator	Stores mandatory or preferred evaluation conditions.
Quotation	file metadata, vendor fields, specifications, source pages, processing and review state	Preserves the uploaded document and reviewed structured evidence.
VendorResult	component scores, overall score, status, failures, risks, missing information, rank	Stores one deterministic comparison result per quotation.

Quotation extraction fields

The reviewed quotation record supports vendor name, product name, model, quantity, unit price, subtotal, tax, total price, currency, delivery days, warranty months, payment terms, support details, validity period, technical specifications, source-page references, and missing-information notes.

6. API Contract

Method	Path	Purpose
GET	/api/health	Health check.
POST	/api/evaluations	Create evaluation and optional requirements.
GET	/api/evaluations	List evaluations.
GET	/api/evaluations/{id}	Read one evaluation.
PATCH	/api/evaluations/{id}	Update evaluation inputs.
DELETE	/api/evaluations/{id}	Delete evaluation and linked data.
POST	/api/evaluations/{id}/requirements	Add one or more requirements.
GET	/api/evaluations/{id}/requirements	List requirements.
POST	/api/evaluations/{id}/quotations	Upload a PDF quotation.
GET	/api/evaluations/{id}/quotations	List quotations.
POST	/api/quotations/{id}/process	Extract PDF text and run Foundry extraction.
PATCH	/api/quotations/{id}	Save reviewed quotation data.
DELETE	/api/quotations/{id}	Delete quotation and invalidate comparison.
POST	/api/evaluations/{id}/evaluate	Run deterministic scoring.
GET	/api/evaluations/{id}/comparison	Return saved vendor comparison.
POST	/api/evaluations/{id}/recommendation	Generate validated recommendation.

7. PDF and AI Processing

PDF validation and storage

- Only .pdf filenames and supported PDF content types are accepted.
- The upload must be non-empty, within MAX_UPLOAD_SIZE_MB, and begin with a PDF header.
- Display filenames are sanitised; stored files use generated UUID names.
- Files are stored under an evaluation-specific directory with path traversal checks.
- PyMuPDF extracts text page by page. Image-only PDFs are rejected because OCR is outside the current scope.

Foundry responsibility 1: structured extraction

The service sends extracted text to the configured deployment through `client.beta.chat.completions.parse`. The prompt treats document text as untrusted data, prohibits invention and estimation, requires null for unclear fields, and records missing information. Pydantic validates the structured response before persistence.

Foundry responsibility 2: recommendation language

After scoring, Foundry receives requirements, reviewed quotations, Python-generated statuses, ranks, scores, risks, and missing information. It returns `recommended_vendor`, `concise_reasoning`, `strengths`, `risks`, `missing_information`, and `tradeoff_explanation`. The API rejects a vendor that is absent or ineligible.

Trust boundary

Foundry interprets text and writes explanations. It does not calculate, replace, or override compliance outcomes, price scores, weighted scores, vendor status, or rank.

8. Deterministic Compliance and Scoring

Requirement evaluation

- Numeric requirements use explicit operators or inferred minimum/maximum semantics.
- Memory units are normalised to GB; duration units are normalised to months or days.
- Missing or incompatible values produce UNKNOWN, never an assumed pass.
- A mandatory FAIL produces NON_COMPLIANT status.
- A mandatory UNKNOWN produces MISSING_INFORMATION unless a mandatory failure already exists.

Price score

$$\text{Price score} = \text{lowest eligible vendor price} / \text{current vendor price} \times 100$$

Eligible price candidates are vendors with COMPLIANT or PARTIALLY_COMPLIANT status and a total price greater than zero. Missing, zero, or invalid prices return a price score of 0. Scores are capped at 100.

Component	Weight
Price	35%
Technical compliance	30%
Delivery	15%
Warranty	10%
Payment terms	5%
Support	5%

Overall score is the weighted sum of the six component scores. Ranking first prioritises status (compliant, partially compliant, missing information, non-compliant), then higher overall score, lower price, and vendor name. Failed vendors remain visible.

9. Validation, Errors, and Security

Condition	API behaviour
Evaluation or quotation not found	404 with a human-readable message.
More than three quotations	409 conflict.
Invalid, empty, oversized, unreadable, or corrupted PDF	422 validation response.
Foundry configuration missing	503 service unavailable.
Foundry request or malformed structured response	502 bad gateway; uploaded file remains available for retry.
Scoring before all quotations are reviewed	409 conflict.
No quotation or no eligible recommendation candidate	409 conflict.
Database write failure	500 with a controlled message and rollback where applicable.

Security controls

- Foundry API keys are backend-only environment secrets.
- NEXT_PUBLIC_API_URL contains only the public backend address.
- CORS allows the configured FRONTEND_URL.
- Pydantic rejects unexpected request fields and validates numeric ranges.
- PDF paths, evaluation identifiers, and stored filenames are constrained.
- Model output is validated and recommendation eligibility is checked in Python.

10. Configuration and Quality Verification

Area	Configuration
Backend	DATABASE_URL, FOUNDRY_ENDPOINT, FOUNDRY_API_KEY, FOUNDRY_MODEL_DEPLOYMENT, FOUNDRY_REQUEST_TIMEOUT_SECONDS, UPLOAD_DIR, FRONTEND_URL, MAX_UPLOAD_SIZE_MB
Frontend	NEXT_PUBLIC_API_URL
Backend checks	uv run pytest
Frontend checks	pnpm typecheck; pnpm lint; pnpm build

Automated coverage

The repository contains tests for scoring, mandatory failures, missing values, price eligibility, the three-vendor scenario, PDF validation, path safety, API workflow gates, deletion behaviour, Foundry endpoint normalisation, structured responses, and recommendation safety. Foundry calls are mocked in automated tests.

11. Reference Demonstration Outcome

Vendor	Price	Key evidence	Expected status
TechCore Solutions	PKR 3,625,000	16 GB RAM, 512 GB SSD, 10-day delivery, 36-month warranty	COMPLIANT and recommended
Digital Systems	PKR 3,450,000	18-day delivery exceeds the mandatory 14-day limit	NON_COMPLIANT
Future Computers	PKR 3,300,000	8 GB RAM and 12-month warranty fail mandatory minimums	NON_COMPLIANT

TechCore Solutions is not hardcoded. It emerges as the recommended vendor because it is the only quotation that satisfies the mandatory evidence in the supplied sample scenario.

12. Known Deployment Constraints

- SQLite and local uploads are suitable for local use and short demonstrations.
- A persistent cloud deployment should use PostgreSQL and external object storage.
- Table creation currently uses SQLAlchemy metadata rather than a migration framework.
- The application has no authentication or tenant isolation.
- AI quality depends on readable source text and the configured Foundry deployment.

Decision-support statement

BidSight provides structured evidence, reproducible scores, and an advisory recommendation. The final purchasing decision remains with the authorised procurement user.